Alexandria University
Faculty of Engineering
Computer and Systems Engineering Dept.
CS242: Data Structures & Algorithms

Sorting Algorithms App Documentation

Name (ID):

1. Mohamed Abdalla Yassen (23010765)

INDEX

1 - **Backend Design and Code Structure**

2 - **Sorting Algorithms Implementation**

3 - **Frontend Design & Cod Structure**

4 - **Design Decisions**

5 - **Sorting Comparison and Data Analysis**

6 - **Sample Output**

7 - **Video “how to use”**

8 - **GitHub Repository**

9 - **Colab**

1- Backend Design and Code Structure

The backend is built using a Spring Boot framework, following a strictly decoupled, layered architecture. To satisfy the two distinct operational modes (Sorting Comparison and Sorting Visualization).

1.1. The REST API (Controller Layer)

The entry point for the frontend is the Controller class, annotated with @RestController.

- **Endpoints:** It defines specific POST mapping endpoints for each visualization algorithm (e.g., /bubble_sort, /quick_sort) and a unified endpoint for the benchmarking mode (/comparison/benchMark).
- **Dependency Injection:** The controller uses constructor injection to safely instantiate the required services.
- **CORS Configuration:** The @CrossOrigin(origins = "http://localhost:4200") annotation is used to allow the Angular frontend development server to communicate seamlessly with the Spring Boot backend.

1.2. Dual-Track Sorting Architecture

Because recording intermediate steps adds memory and processing overhead, using the same sorting method for both visualization and benchmarking would result in inaccurate time measurements. To solve this, the application splits the logic into two distinct packages:

A. The Visualization Engine (com.assignment1.sorting_algo.service) These services are optimized for state-tracking rather than speed.

- **Step Generation:** Algorithms in this package return a List<Step>. The Step DTO acts as a snapshot, recording the step_num, operation (swap or compare), the current array state, and pointer indices (ptr1, ptr2).
- **Visual Accuracy:** This ensures the Angular frontend receives exact, frame-by-frame data to animate the bar charts changing in real-time.

B. The Benchmarking Engine (com.assignment1.sorting_algo.service_comparator) These classes (e.g., Bubble, Quick, Merge) are optimized for raw performance and data collection to satisfy the "Sorting Comparison" requirements.

- **Precision Timing:** They calculate execution time using System.nanoTime() for high-resolution performance metrics.
- **Multiple Runs & Data Export:** Each comparator class contains a multipleRuns method that executes the algorithm a specified number of times. Crucially, it exports the raw execution times to CSV files (e.g., benchmark_results_merge.csv) utilizing PrintWriter and FileWriter.
- **Aggregation Engine:** The RunAllMethods service acts as an orchestrator. It iterates through all sorting algorithms, triggers their multipleRuns methods, and packages the aggregated statistics into a list of BenchMarkDTO objects.

1.3. Data Transfer Objects (DTOs)

The application utilizes distinct DTOs to maintain clean boundaries between the backend and frontend:

- **RequestDTO:** The universal input model from the frontend. It dictates the size of the array, the number of runs, and the order (1 for sorted, 2 for reverse-sorted, 3 for random, 4 for file input).
- **UnitRun:** A granular metric object used internally by the comparator services to track comparisons, swaps, and runTime for a single execution.
- **BenchMarkDTO & DataOfMethod:** Used to format the final output for the comparison table. BenchMarkDTO holds the metadata (algorithm name, array size), while DataOfMethod contains the statistical calculations (minTime, maxTime, avgTime, total comparisons, and swaps).

1.4. Data Generation Utility

The Helper class centralizes array manipulation to ensure consistency across all tests. Based on the order parameter from the RequestDTO, it generates arrays that are random, fully sorted, or inversely sorted.

2- Sorting Algorithms Implementation

The application implements six distinct comparison-based sorting algorithms. Below is an explanation of the logic and the pseudocode for each, based on the specific implementations in the backend engine.

2.1. Selection Sort

Selection Sort divides the array into two parts: the sorted part at the left end and the unsorted part at the right end. It repeatedly selects the smallest (minimum) element from the unsorted subarray and swaps it with the leftmost unsorted element.

Pseudocode:

```
function selectionSort(array):
      n = length(array)
      for i from 0 to n - 2:
      min_idx = i
      for j from i + 1 to n - 1:
            increment comparisons
            if array[min_idx] > array[j]:
                  min_idx = j

      if min_idx != i:
            increment interchanges
            swap(array[min_idx], array[i])
```

2.2. Bubble Sort

Bubble Sort works by repeatedly stepping through the list, comparing adjacent elements, and swapping them if they are in the wrong order. Your implementation includes an important optimization: it uses a boolean flag (`swapped`) to stop the algorithm early if the array becomes sorted before all iterations are complete.

Pseudocode:

```
function bubbleSort(array):
      n = length(array)
      for i from 0 to n - 2:
         swapped = false
         for j from 0 to n - i - 2:
            increment comparisons
            if array[j] > array[j + 1]:
                  increment interchanges
                  swap(array[j], array[j + 1])
                  swapped = true

      if swapped is false:
            break // Array is already sorted
```

2.3. Insertion Sort

Insertion Sort builds the final sorted array one item at a time. It iterates through the input array, takes one element, and finds its correct location within the already-sorted elements to its left.

Pseudocode

```
function insertionSort(array):
      n = length(array)
      for i from 1 to n - 1:
            for j from i down to 1:
                  increment comparisons
                  if array[j] < array[j - 1]:
                        increment interchanges
                        swap(array[j], array[j - 1])
                  else:
                        break
```

2.4. Merge Sort

Merge Sort is a Divide and Conquer algorithm. It divides the input array into two halves, calls itself recursively for the two halves, and then merges the two sorted halves. Your implementation carefully tracks elements overwritten during the merge phase as "swaps" to maintain metric consistency.

Pseudocode:

```
function mergeSort(array, left, right):
    if left < right:
        mid = left + (right - left) / 2
        mergeSort(array, left, mid)
        mergeSort(array, mid + 1, right)
        merge(array, left, mid, right)

function merge(array, left, mid, right):
    L = array[left ... mid]
    R = array[mid + 1 ... right]
    i = 0, j = 0, k = left

    while i < length(L) and j < length(R):
        increment comparisons
        if L[i] <= R[j]:
            array[k] = L[i]
            i = i + 1
        else:
            array[k] = R[j]
            j = j + 1
        k = k + 1
        increment swaps

    // Copy remaining elements
    while i < length(L):
        array[k] = L[i]
        i = i + 1; k = k + 1; increment swaps

    while j < length(R):
        array[k] = R[j]
        j = j + 1; k = k + 1; increment swaps
```

2.5. Quick Sort

Quick Sort also uses the Divide and Conquer strategy. Your implementation uses the Lomuto partition scheme, picking the last element as the pivot (`pivotVal = array.get(r)`) and partitioning the other elements into two sub-arrays according to whether they are less than or greater than the pivot.

Pseudocode:

```
function quickSort(l, r, array):
    if l >= r:
        return

    pivotVal = array[r]
    i = l - 1

    for j from l to r - 1:
        increment comparisons
        if pivotVal > array[j]:
            i = i + 1
            increment swaps
            swap(array[j], array[i])

    increment swaps
    swap(array[r], array[i + 1])
    pivot_idx = i + 1

    quickSort(l, pivot_idx - 1, array)
    quickSort(pivot_idx + 1, r, array)
```

2.6. Heap Sort

Heap Sort is a comparison-based sorting technique based on a Binary Heap data structure. First, your code builds a "Max Heap" from the input data, meaning the largest item is stored at the root node. Then, it repeatedly swaps the root with the last item of the heap and reduces the heap size, calling `heapify` to restore the heap property.

Pseudocode:

```
function heapSort(array):
    n = length(array)
    buildMaxHeap(array)

    for i from n - 1 down to 1:
        swap(array[0], array[i])
        increment swaps
        heapify(0, i, array)

function buildMaxHeap(array):
    n = length(array)
    for i from (n / 2) - 1 down to 0:
        heapify(i, n, array)

function heapify(node_idx, size, array):
    max_idx = node_idx
    l_child = node_idx * 2 + 1
    r_child = node_idx * 2 + 2

    if l_child < size and array[max_idx] < array[l_child]:
        increment comparisons
        max_idx = l_child

    if r_child < size and array[max_idx] < array[r_child]:
        increment comparisons
        max_idx = r_child

    if max_idx != node_idx:
        swap(array[max_idx], array[node_idx])
        increment swaps
        heapify(max_idx, size, array)
```

3- Frontend Design & Cod Structure

The Graphical User Interface (GUI) is built using **Angular**, taking advantage of its latest features like reactive Signals and modern control flow syntax. The frontend is designed to seamlessly support the two required modes of operation: Sorting Comparison and Sorting Visualization.

4.1. Centralized API Communication

To maintain a clean separation of concerns, all HTTP requests to the Spring Boot backend are centralized in a single injectable service, SortService.

- **Endpoints:** It defines specific POST methods returning RxJS Observable streams for each sorting algorithm's visualization data (e.g., getSortSteps_bubble, getSortSteps_quick) and a dedicated method for the benchmarking engine (getBenchmarkData).
- **Shared Interfaces:** It exports the requestBody interface, ensuring strict typing for the parameters (size, order, runs) sent to the backend.

4.2. Sorting Comparison (Benchmarking) Interface

The Comparator component handles the "Sorting Comparison" requirement.

- **Dynamic Inputs:** The HTML template provides a form bound via [(ngModel)] to capture user inputs for array size, number of runs, and generation mode.
- **File Upload Handling:** If the user selects the "FromFile" generation mode (order 4), an @if block conditionally renders a file upload input. The onFileSelected method utilizes a FileReader to extract the text content from the uploaded .txt file.
- **Results Table:** Once the benchmark runs, the returned BenchMarkDTO data is stored in an Angular Signal (Data = signal<any[]>([])). An @for loop dynamically generates a table row for each algorithm, formatting the nanosecond execution times using Angular's decimal pipe (number:'1.0-2') for readability.

4.3. Sorting Visualization Engine

All six sorting algorithms share a unified visualization architecture, modeled here using the BubbleSort component as the blueprint.

A. Reactive State Management (Signals) Because the visualization requires rendering arrays frame-by-frame, Angular Signals are used for high-performance state tracking.

- The steps signal stores the entire array of snapshot data returned from the backend.
- Signals like currentArray, ptr1, ptr2, and isSwapping are updated at each step to trigger immediate, precise UI re-renders without relying on heavy DOM manipulation.

B. Dynamic Bar Rendering The visualization represents the array values as vertical bars.

- **Height Calculation:** The height of each bar is calculated dynamically as a percentage of the maximum possible value: [style.height.%]="(value / maxVal) * 100".
- **Visual Indicators:** CSS classes are dynamically bound to highlight the active elements. If an element's index matches ptr1 or ptr2, it gets the .comparing class. If the step's operation is a swap, it gets the .swapping class.

C. Playback Controls The component offers granular control over the visualization playback:

- **Manual Stepping:** nextStep() and prevStep() functions allow the user to manually traverse the steps array, updating the pointers and running totals for comparisons and interchanges on demand.
- **Automated Playback:** The autoRun() function utilizes an asynchronous while loop with await new Promise(resolve => setTimeout(resolve, this.speed)). This elegantly pauses the execution based on the user-defined speed slider, creating a smooth animation effect.

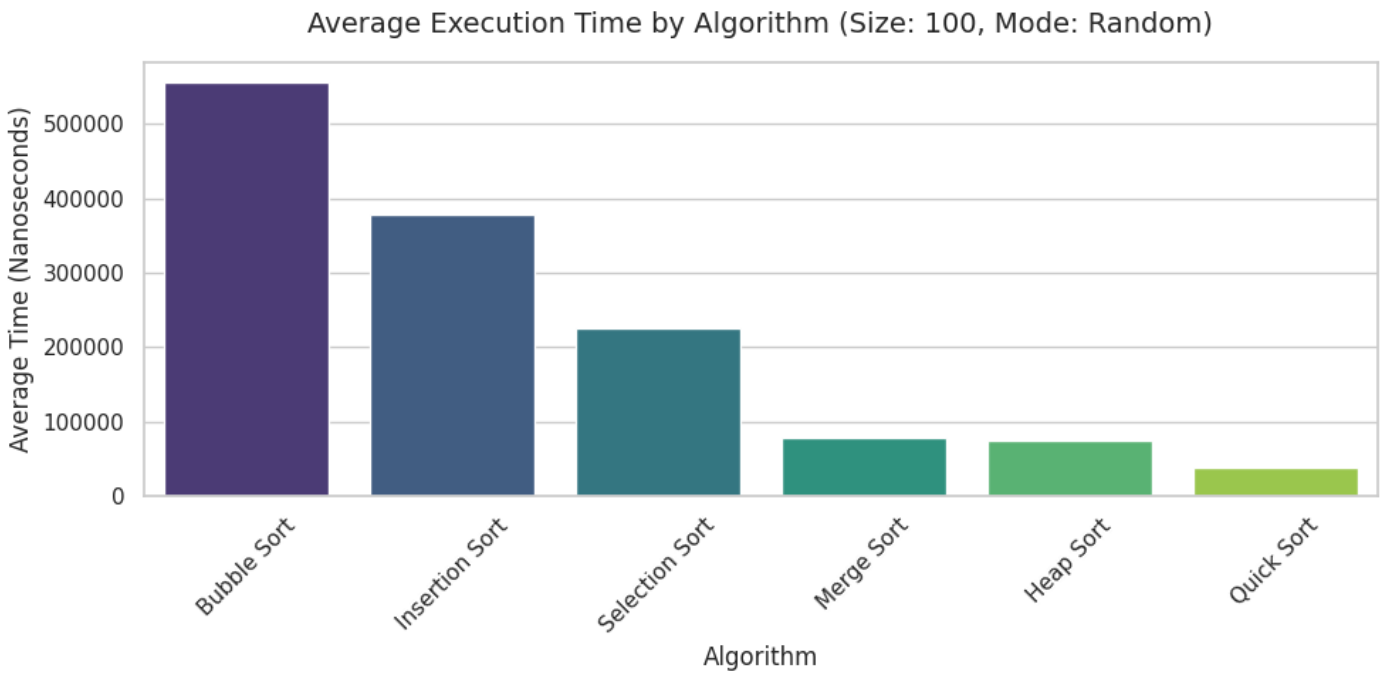
4- Design Decisions

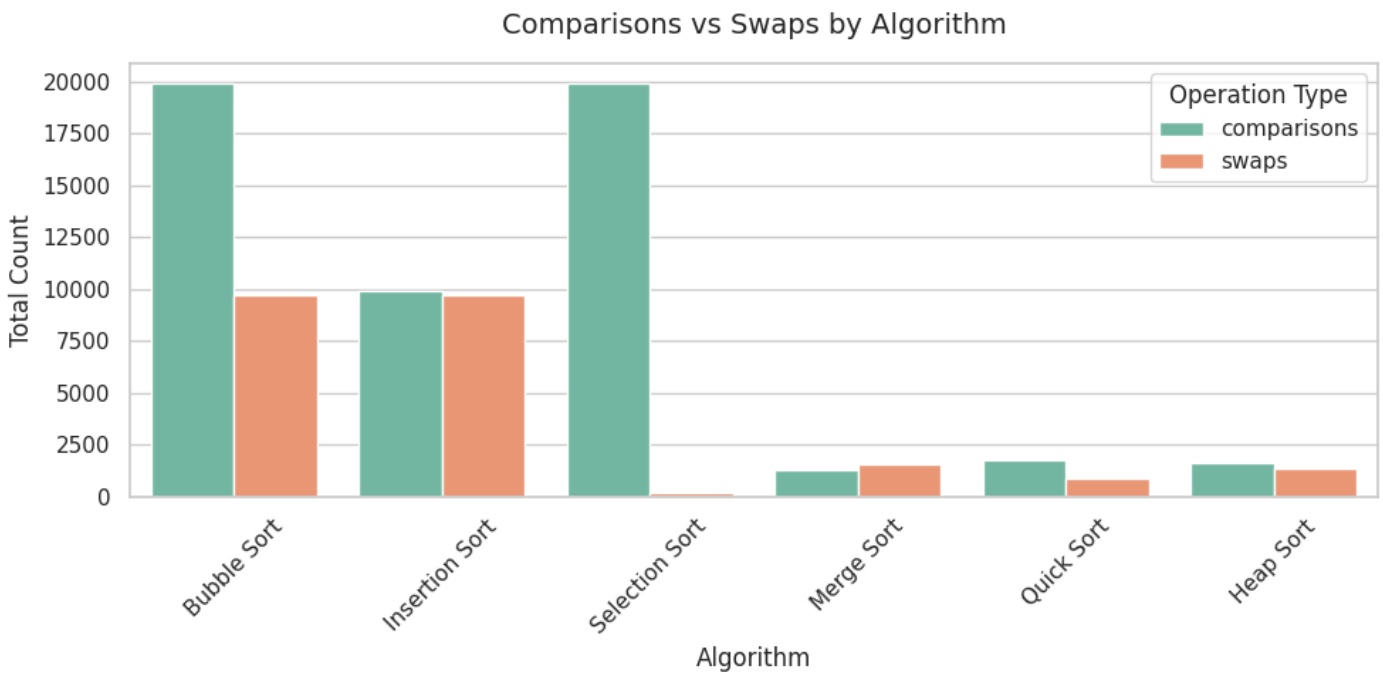
1. Sending All Steps at Once (Client-Side Playback)
   - The Problem:
     If the frontend had to ask the backend for every single frame of the animation, the app would lag and stutter due to constant network requests.
   - The Solution:
     The backend calculates the *entire* sorting process instantly and sends a complete list of steps (List<Step>) to the frontend in one go. This allows the frontend to smoothly play, pause, or step backward through the animation without talking to the server again.

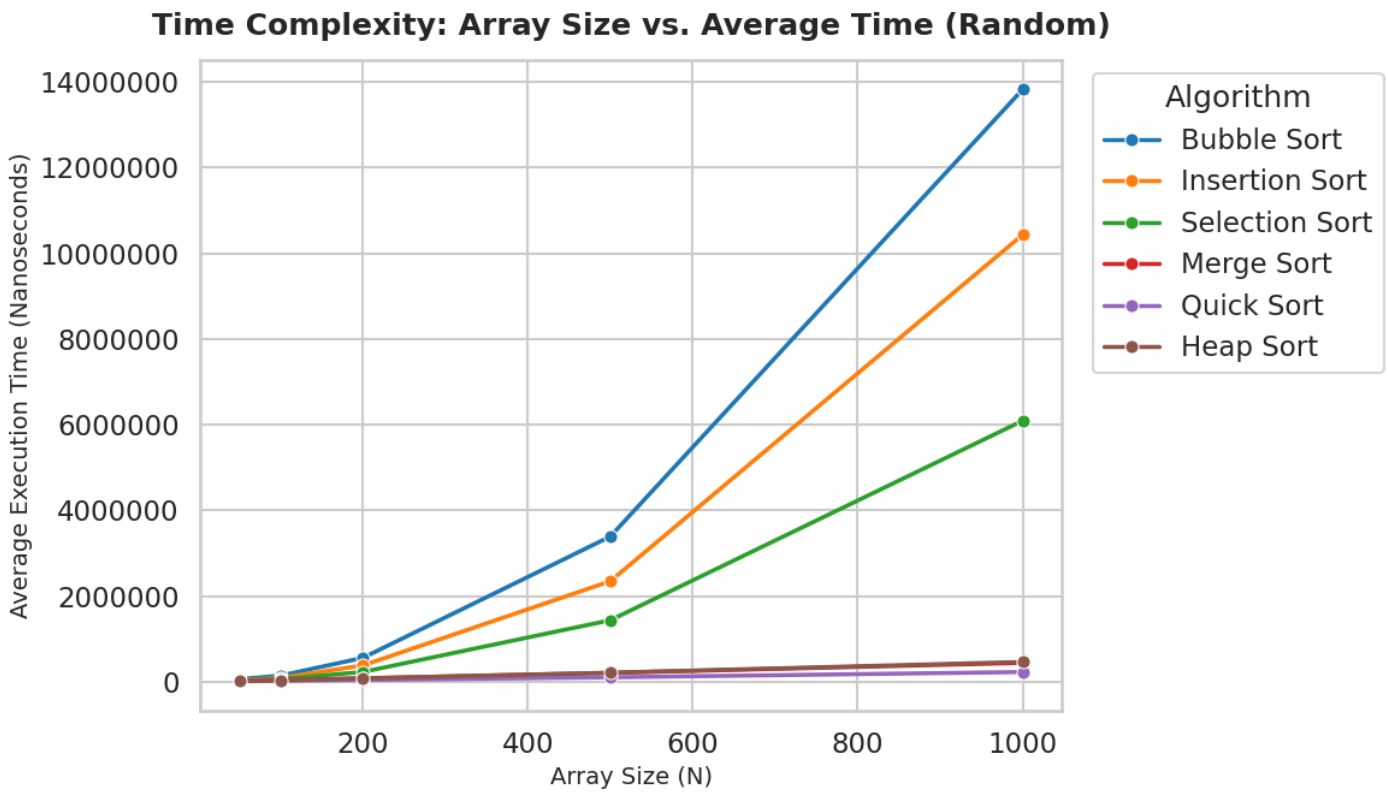
2. Smart Screen Updates (Angular Signals)
   - The Problem:
     Sorting 100 items can generate thousands of steps. Redrawing the entire screen thousands of times per second would crash the browser.
   - The Solution:
     I used Angular's modern signal() feature. This tells the browser to *only* redraw the specific bars that are actively swapping or comparing, rather than refreshing the whole page. This keeps the animations incredibly fast and smooth.
3. Simulating Swaps for Unified Visualization
   - The Problem:
     Some algorithms, like Merge Sort, do not traditionally "swap" elements in place; instead, they overwrite values from temporary arrays. However, the frontend visualizer was built to animate standard "swaps" and "comparisons".
   - The Solution:
     I adapted the backend logic for these algorithms to record their index overwrites as "swaps" (by having both pointers point to the same updated index). This clever design decision meant that all six algorithms could share the exact same UI visualization component, preventing the need to write custom animation logic for each algorithm.

5- Sorting Comparison and Data Analysis

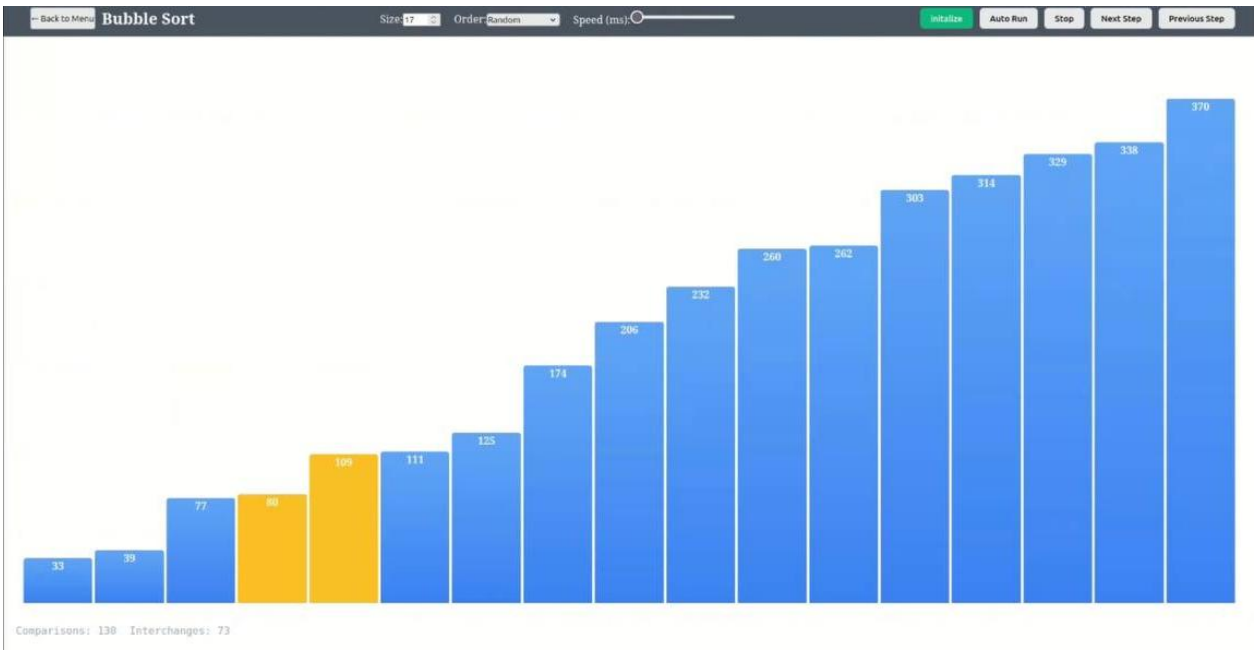
Algorithm	Best Case	Average Case	Worst Case
Bubble Sort	$O(N)$	$O(N^2)$	$O(N^2)$
Selection Sort	$O(N^2)$	$O(N^2)$	$O(N^2)$
Insertion Sort	$O(N)$	$O(N^2)$	$O(N^2)$
Merge Sort	$O(N\ logN)$	$O(N\ logN)$	$O(N\ logN)$
Quick Sort	$O(N\ logN)$	$O(N\ logN)$	$O(N^2)$
Heap Sort	$O(N\ logN)$	$O(N\ logN)$	$O(N\ logN)$

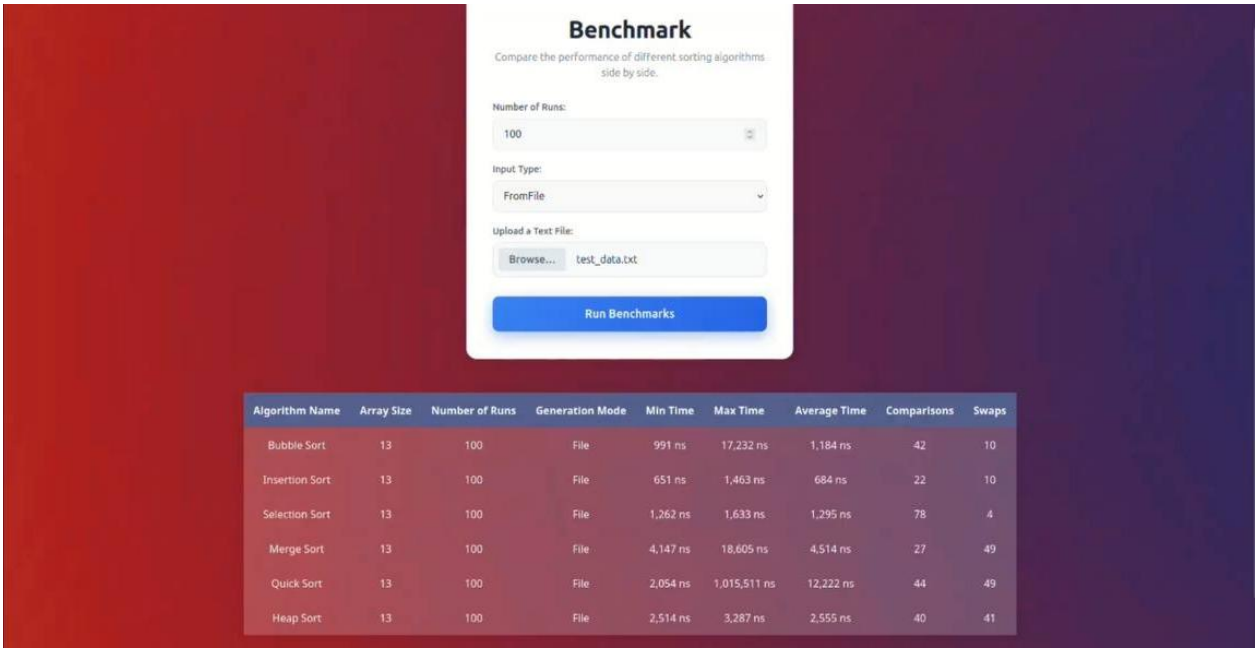






6- Sample Output





Algorithm Name	Array Size	Number of Runs	Generation Mode	Min Time	Max Time	Average Time	Comparisons	Swaps
Bubble Sort	13	100	File	991 ns	17,232 ns	1,184 ns	42	10
Insertion Sort	13	100	File	651 ns	1,463 ns	684 ns	22	10
Selection Sort	13	100	File	1,262 ns	1,633 ns	1,295 ns	78	4
Merge Sort	13	100	File	4,147 ns	18,605 ns	4,514 ns	27	49
Quick Sort	13	100	File	2,054 ns	1,015,511 ns	12,222 ns	44	49
Heap Sort	13	100	File	2,514 ns	3,287 ns	2,555 ns	40	41

7- Video “how to use”

https://drive.google.com/file/d/1umnsGn5yq0nRsM3cj214FSlQjC2owlkZ/view?usp=drivesdk

8- GitHub Repository

https://github.com/Mohamed-Abdalla-Yassen/Sorting-algorithms.git

9- Colab

https://colab.research.google.com/drive/1x8vFNpLd38E7T64pqw7jvX2r4qmbnYOG?usp=sharing